



Escuela Técnica Superior de
Ingeniería Informática

Trabajo Complementos de Bases de Datos

DeckBuilder una aplicacion en C con MongoDB

Realizado por

**Borrego González, Adolfo
Sánchez Carmona, Germán**

Grado

Grado en Ingeniería Informática - Ingeniería del Software

Departamento de

Lenguajes y Sistemas Informáticos

Índice general

1	Ejemplos de uso de LaTeX	1
1.1.	Introducción	1
1.2.	Estilos	1
1.3.	Listados	1
1.4.	Subsecciones	2
1.4.1.	Primera subsección	2
1.4.2.	Segunda subsección	2
1.5.	Imágenes y figuras	2
1.6.	Tablas	3
1.7.	Referencias	4
1.8.	Extractos de código	4
1.9.	Enlaces	6
1.10.	Citas y bibliografía	6
1.11.	Ecuaciones	6
1.12.	Caracteres y símbolos especiales	7
2	Introducción	8
3	Planificación y Diseño	9
3.1.	Ideas y Alcance	9
3.2.	Análisis del Flujo de Usuario y Casos de Uso	9
3.3.	Diseño de la Arquitectura de Datos en MongoDB	10
3.4.	Stack Tecnológico	10
3.5.	Resultado de la fase	10
4	Desarrollo CORE	11
4.1.	Implementación del núcleo funcional	11
4.2.	Desarrollo del backend y consolidación del CRUD	11
4.3.	Desarrollo del frontend y flujo operativo real	13
4.4.	Capacidades multijuego y editor visual de baraja	13
4.5.	Resultados de la fase	14
5	Memoria Técnica y Presentación	15
5.1.	Elaboración de la documentación técnica	15
5.2.	Preparación de la presentación	15
5.3.	Desarrollo del archivo README: Manual usuario y despliegue	15
5.4.	Resultados finales de la fase	16
6	Conclusiones	17
	Bibliografía	18

1. Ejemplos de uso de LaTeX

Este capítulo se incluye únicamente como ayuda y referencia de uso de $\text{\LaTeX}$. No debe aparecer en el documento final.

1.1. Introducción

En este capítulo se muestran ejemplos de uso de $\text{\LaTeX}$ para operaciones comunes.

1.2. Estilos

Se pueden aplicar estilos al texto como **negritas**, *cursiva*, subrayado y monoespaciado. También se **pueden** aplicar **colores**, y combinar **estilos**. Se recomienda usar sólo negritas para hacer énfasis, y no abusar de este recurso.

Por comodidad para usuarios no habituados con LaTeX, esta plantilla define algunos alias de comandos más fáciles de recordar para estilos de texto comunes: **negritas**, *cursiva* y código.

1.3. Listados

Con itemize se pueden crear listas no numeradas:

- Fresas
- Melocotones
- Piñas
- Nectarinas

De manera similar, enumerate permite crear listas numeradas:

1. Elaborar la memoria del TFG
2. Elaborar la presentación
3. Presentar el TFG
4. Solicitar el título de Grado

1.4. Subsecciones

Se pueden definir subsecciones con el comando `subsection`:

1.4.1. Primera subsección

Esto es una subsección

1.4.2. Segunda subsección

Esto es otra subsección.

Sub-sub-sección

Este es un tercer nivel de profundidad, que no aparece en el índice general. Se recomienda no utilizarlo, si es posible.

1.5. Imágenes y figuras

Todas las imágenes y figuras del documento se incluirán en la carpeta “figures”. Se pueden incluir de la siguiente manera:



Figura 1.1: Un feroz depredador

Observe que las figuras se numeran automáticamente según el capítulo y el número de figuras que hayan aparecido anteriormente en dicho capítulo. Existen muchas maneras de definir el tamaño de una figura, pero se aconseja utilizar la mostrada en este ejemplo: se define el ancho de la figura como un porcentaje del

ancho total de la página, y la altura se escala automáticamente. De esta manera, el ancho máximo de una figura sería $1.0 * \text{textwidth}$, lo que aseguraría que se muestra al máximo tamaño posible sin sobrepasar los márgenes del documento.

Tenga en cuenta que LaTeX intenta incluir las figuras en el mismo sitio donde se declaran, pero en ocasiones no es posible por motivos de espacio. En esos casos, LaTeX colocará la figura lo más cerca posible de su declaración, puede que en una página diferente. Esto es un comportamiento normal.

1.6. Tablas

Existe una gran variedad de formas de crear tablas en LaTeX puro, y todas ellas tienen cierta complejidad. A continuación se muestra un ejemplo simple de tabla nativa, en la Tabla 1.1. Se recomienda crear un archivo en la carpeta *tables* por cada tabla nativa que se desee incluir, y enlazarla mediante el comando `input`.

Columna L	Columna C	Columna R	Columna P
Texto de ejemplo	Texto de ejemplo	Texto de ejemplo	Texto de ejemplo
ABC	DEF	HIJ	KLM

Tabla 1.1: Tabla LaTeX de ejemplo

Para tablas con un formato más complejo, considere la posibilidad de diseñarla usando otro software externo (por ejemplo Excel) e incluirla de manera similar a una figura. **Observe en el código LaTeX a continuación cómo usar el comando `captionof{table}`, en lugar de simplemente `caption`, hace que se liste como una Tabla en lugar de como una Figura:**

		%			Comparisons		
Group	N	positive	neutral	negative	χ^2	df	p
Q1 (a)	Programming Labs are useful for learning the subject						
CS group	400	73	7	20	0.9	4	ns
Math group	400	70	8	22			
Q1 (b)	Working with data makes predictions more reliable						
CS group	400	52	12	36	13.7	4	< 0.05
Math group	400	44	15	41			
Q1 (c)	Lessons in class are more active than web conferences						
CS group	400	46	11	43	14.8	4	< 0.01
Math group	400	31	14	55			

Tabla 1.2: Tabla compleja introducida como figura

1.7. Referencias

Observe cómo en el código fuente de esta sección se ha usado varias veces el comando `label`. Este comando permite marcar un elemento, ya sea capítulo, sección, figura, etc. para hacer una referencia numérica al mismo. Para referenciar una label se usa el comando `ref` incluyendo el nombre de la referencia:

Este es el capítulo [1](#).

En la sección [1.2](#) se muestran ejemplos de estilos.

La subsección [1.4.1](#) explica...

En la Figura [1.1](#) vemos que...

Esto evita que tengamos que escribir directamente los índices de las secciones y figuras que queremos mencionar, ya que LaTeX lo hace por nosotros y además se encarga de mantenerlos actualizados en caso de que cambien (pruebe a mover este capítulo al final del documento y observe cómo se actualizan automáticamente todos los índices referenciados). Además, las referencias mediante “`ref`” actúan como hipervínculos dentro del documento que llevan al elemento referenciado al pulsar en ellas.

Es habitual nombrar las “label” con un prefijo que indica el tipo de elemento para encontrarlo luego más fácilmente, pero no es obligatorio.

1.8. Extractos de código

Se pueden incluir extractos de código mediante `lstlisting`:

```
1 num = float(input("Enter a number: "))
2 if num > 0:
3     print("Positive number")
4 elif num == 0:
5     print("Zero")
6 else:
7     print("Negative number")
```

Extracto de código 1.1: Código Python

Para evitar tener que incluir el código directamente en el texto del documento, se pueden guardar en archivos separados y referenciarlos:

Los extractos de código también se pueden referenciar mediante `label/ref`: Extractos de código [1.1](#), [1.2](#), [1.3](#), [1.4](#).

```
1 class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("Hello, LaTeX!");  
4     }  
5 }
```

Extracto de código 1.2: Código Java

```
1 <html>  
2     <head>  
3         <title>LaTeX example</title>  
4     </head>  
5  
6     <body>  
7         Hello, LaTeX!  
8     </body>  
9 </html>
```

Extracto de código 1.3: Código HTML

```
1 function main() {  
2     console.log("Hello, LaTeX!");  
3 }
```

Extracto de código 1.4: Código JavaScript

1.9. Enlaces

Puede enlazar una web externa mediante el comando `url`: <https://www.example.com>. También se puede vincular un enlace a un texto mediante el comando `href`: [dominio de ejemplo](#).

1.10. Citas y bibliografía

En LaTeX, los elementos de la bibliografía se almacenan en un fichero bibliográfico en un formato llamado BibTeX, en el caso de este proyecto se encuentran en “bibliografia.bib”. Para citar un elemento se usa el comando `cite`. Se pueden citar tanto artículos científicos [1] como enlaces web [2].

También se puede usar el comando `citet` para incluir una referencia junto con el nombre de su autor o autores: Borrego et al. [3]. Todas las citas se numeran automáticamente y se incluyen en la sección de bibliografía del trabajo. El orden por defecto es según su orden de aparición en el documento. Para ordenarlas por orden alfabético del autor, puede modificar el comando `bibliographystyle` del archivo principal y reemplazar su valor por el estilo `plainnat` (orden alfabético, nombres completos) o `abbrvnat` (orden alfabético, nombres abreviados).

Observe cómo los elementos bibliográficos almacenados en “bibliografia.bib” tienen una etiqueta asociada, que es la que se usa al citarlos mediante `cite`. **Añadir una referencia al fichero bibliográfico no hace que ésta aparezca automáticamente en la sección de bibliografía del trabajo, es necesario citarla en algún lugar del mismo.**

1.11. Ecuaciones

LaTeX tiene un potente motor para mostrar ecuaciones matemáticas y un amplio catálogo de símbolos matemáticos. El entorno matemático se puede activar de muchas maneras. Para incluir ecuaciones simples en un texto se pueden rodear de símbolos dólar: $1 + 2 = 3$, $\sqrt{81} = 3^2 = 9$, $\forall x \in y \exists z : S_z < 4$.

Las ecuaciones más complejas pueden expresarse aparte y son numeradas: ecuación 1.1.

$$\lim_{x \rightarrow 0} \frac{e^x - 1}{2x} \stackrel{\begin{smallmatrix} 0 \\ 0 \end{smallmatrix}}{H} \lim_{x \rightarrow 0} \frac{e^x}{2} = \frac{1}{2} + 7 \int_0^2 \left(-\frac{1}{4} (e^{-4t_1} + e^{4t_1-8}) \right) dt_1 \quad (1.1)$$

Dispone [aquí](#) de un amplio listado de símbolos que pueden usarse en modo matemático.

1.12. Caracteres y símbolos especiales

Algunos caracteres y símbolos deben ser escapados para poder representarse en el documento, ya que tienen un significado especial en LaTeX. Algunos de ellos son:

- El símbolo dólar \$ se usa para ecuaciones.
- El tanto por ciento % se usa para comentarios en el código fuente.
- El símbolo euro € suele dar problemas si se escribe directamente.
- El guión bajo _ se usa para subíndices en modo matemático.
- Las comillas deben expresarse ‘así’ para comillas simples y “así” para comillas dobles. Las comillas españolas pueden expresarse «así».
- La barra invertida o contrabarra \ se usa para comandos LaTeX.
- Otros símbolos que deben escaparse son las llaves { }, el ampersand &, la almohadilla # y los símbolos mayor que > y menor que <.

2. Introducción

En los últimos años ha crecido notablemente la popularidad de los juegos de cartas coleccionables (TCG) como Pokémon, Magic: The Gathering y Yu-GI-Oh! Esto ha generado una demanda creciente de herramientas digitales para la gestión de inventarios y la construcción de barajas competitivas. En este contexto, este trabajo describe el diseño e implementación de Deck Builder, una aplicación orientada a centralizar y optimizar la experiencia del coleccionista.

El núcleo técnico de este proyecto consiste en el desarrollo de un sistema CRUD (Create, Read, Update, Delete) robusto, implementado en C# sobre el entorno .NET 8 y utilizando MongoDB como motor de base de datos. La elección de estas tecnologías responde a la necesidad de gestionar datos heterogéneos, con estructuras y atributos variables según el juego.

El objetivo principal es ofrecer una plataforma que no solo permita registrar cartas, sino también validar las reglas específicas de cada juego, integrando datos reales obtenidos a través de APIs externas dedicadas para garantizar la precisión de la información. De este modo, se busca reducir el problema de navegar entre múltiples aplicaciones para gestionar las colecciones de diferentes franquicias.

La metodología de trabajo se estructuró en tres fases: planificación y diseño, desarrollo del sistema y, finalmente, la consolidación técnica y documentación del sistema. Este enfoque iterativo ha permitido abordar el proyecto de forma progresiva y segura, asegurando el cumplimiento de los requisitos en cada etapa.

Como resultado, se ha obtenido una aplicación funcional que destaca por su capacidad de adaptarse a distintos esquemas de datos gracias al uso de bases de datos documentales.

3. Planificación y Diseño

3.1. Ideas y Alcance

Esta fase inicial fue determinante para establecer los cimientos de este proyecto. El primer hito fue una reunión inicial de definición de alcance donde se determinó que, para que el sistema fuera útil, debía ser capaz de manejar distintos juegos de cartas sin reestructurar la base de datos para cada franquicia.

Durante la reunión, contemplamos diversas formas de orientar el proyecto sobre la base de las tecnologías escogidas. Tras analizar cómo usar APIs externas para la extracción de cartas, llegamos a la conclusión de que la creación de barajas personalizadas era la idea con más sentido para implementar el CRUD completo que se nos pedía. La visión era crear una base sólida que permitiera, en las siguientes fases, mejorar la lógica interna de validación de reglas.

3.2. Análisis del Flujo de Usuario y Casos de Uso

Para que la aplicación sea operativa, se definió un flujo de usuario lineal y lógico que maximiza la eficiencia en la gestión de colecciones. Este flujo se documentó bajo los siguientes pasos:

- **Identificación y Autenticación:** El usuario accede al sistema, lo que permite vincular sus barajas personalizadas a su perfil único almacenado en MongoDB.
- **Consulta y Descubrimiento:** El usuario utiliza el buscador. Este componente realiza peticiones a la API externa de `tcgdex.dev`, devolviendo el listado de cartas candidatas.
- **Persistencia en Colección:** Una vez seleccionada una carta, el sistema CRUD realiza una operación de creación (CREATE) para guardar la referencia y los metadatos específicos.
- **Construcción de Barajas:** El usuario crea un contenedor “Baraja” donde añade y retira cartas. Aquí se aplica la lógica de edición (UPDATE) para modificar cantidades o estados.
- **Ver y Eliminar:** El usuario puede ver sus barajas, lo que sería la funcionalidad de (READ), y podría, si quisiera, eliminarlas con la operación de (DELETE).

3.3. Diseño de la Arquitectura de Datos en MongoDB

Al haber seleccionado MongoDB por el interés del equipo en las bases de datos no relacionales, el diseño de la arquitectura fue un punto clave. A diferencia de un modelo SQL rígido, optamos por un esquema basado en documentos BSON, lo que permite una flexibilidad total en los atributos.

Figura 1. Modelo datos User (MongoDB)

Figura 2. Modelo datos Deck (MongoDB)

Figura 3. Modelo datos User_Card (MongoDB)

3.4. Stack Tecnológico

El requisito principal del proyecto era construir una API REST que permitiese trabajar con una base de datos NoSQL en MongoDB. Para este proyecto en concreto, teníamos la restricción de que la tecnología a utilizar debía ser C#, por lo que construimos el backend en .NET 8. Este se encarga de la lógica de negocio y exponer la API.

Por otro lado, para ampliar el alcance inicial, se decidió incluir una capa frontend mediante React + Vite + TypeScript, con el objetivo de facilitar el uso de la API de cara a los usuarios.

En cuanto a infraestructura, el proyecto se desplegó en servicios cloud:

- Render para backend y frontend.
- MongoDB Atlas para alojar la base de datos en entorno gestionado.

De esta forma, la solución mantiene el núcleo requerido (MongoDB + C#) y añade una arquitectura web completa, preparada para uso real y validación en producción.

3.5. Resultado de la fase

La fase de planificación y diseño ha permitido establecer una base sólida para el desarrollo del proyecto. Durante esta etapa se han definido claramente los objetivos del sistema, el alcance funcional y el flujo de usuario, asegurando que la aplicación cubra las necesidades principales de gestión de barajas.

Como resultado de esta fase, se ha obtenido una visión clara del funcionamiento global del sistema, incluyendo la interacción entre usuario, aplicación y fuentes externas de datos. Además, ha quedado definido el stack tecnológico. Esto ha facilitado la transición hacia la fase de desarrollo, donde se ha podido implementar el sistema de manera más eficiente y con menor riesgo de errores estructurales.

4. Desarrollo CORE

4.1. Implementación del núcleo funcional

Esta segunda etapa del proyecto se centró en la transformación de los diseños y esquemas previos en un producto de software funcional. El objetivo primordial consistió en construir una base tecnológica sólida que cubriera el ciclo de vida completo del usuario dentro de la aplicación: desde el proceso de autenticación inicial hasta la gestión avanzada de sus barajas.

Para asegurar la calidad del software, la implementación se llevó a cabo de forma incremental y modular. Este enfoque, basado en el cumplimiento de hitos concretos y el uso de commits atómicos, permitió validar de manera constante cada funcionalidad antes de proceder a la siguiente, minimizando el riesgo de errores estructurales.

4.2. Desarrollo del backend y consolidación del CRUD

En el entorno de backend, se estructuró una API bajo el entorno de .NET 8 diseñada para soportar operaciones transaccionales sobre las barajas de usuarios autenticados. Durante este proceso, se priorizó la separación de responsabilidades mediante el uso de DTOs y validaciones independientes, lo que mejora la mantenibilidad del código y evita el acoplamiento entre la lógica de negocio y la capa de transporte de datos.

Sobre esta arquitectura se consolidó el sistema CRUD completo para la entidad de barajas: creación (CREATE), lectura (READ), actualización (UPDATE) y borrado (DELETE). La lógica de actualización fue algo más compleja, ya que incluye la persistencia dinámica de cartas dentro de cada mazo y la implementación de reglas de dominio específicas, como el control de copias máximas permitidas y el tamaño total de la baraja.

Para garantizar la estabilidad de los datos externos se optimizó la comunicación con los proveedores de cartas mediante el desarrollo de un mejor cliente HTTP. Se realizaron ajustes pequeños en los algoritmos de búsqueda para asegurar que las respuestas obtenidas desde fuentes como Pokémon o Magic fueran consistentes y estuvieran listas para su procesamiento en el servidor.

Figura 4. Modelos de Deck y de User (C#)

Figura 5. Ejemplo controlador (C#)

Figura 6. Ejemplo validaciones (C#)

La API desarrollada sigue un diseño REST, estructurado en torno a recursos como usuarios, barajas y cartas. Esta organización permite una separación clara de responsabilidades y facilita la escalabilidad del sistema.

Los endpoints implementados cubren tanto la autenticación como la gestión completa de barajas y la colección de cartas del usuario.

Autenticación

- POST /api/auth/register: registro nuevos usuarios.
- POST /api/auth/login: autenticación de usuarios existentes.

Gestión de barajas (Decks)

- GET /api/decks: obtención de todas las barajas del usuario autenticado.
- GET /api/decks/{deckId}: consulta detallada de baraja específica.
- POST /api/decks: creación de nueva baraja.
- PUT /api/decks/{deckId}: actualización de una baraja, incluyendo modificar cartas.
- DELETE /api/decks/{deckId}: eliminación de una baraja.

Gestión de colección de cartas (User Cards)

- GET /api/user-cards: obtención de colección de cartas del usuario (README, detalles).
- GET /api/user-cards/stats: obtención de estadísticas de colección (README, detalles).
- POST /api/user-cards: inserción de nuevas cartas en colección.
- POST /api/user-cards/{userCardId}/quantity: ajuste incremental de copias carta.
- DELETE /api/user-cards/{userCardId}: eliminación carta de colección.

Búsqueda de cartas

- GET /api/cards/search: búsqueda de cartas fuentes externas.

Este conjunto de endpoints constituye la base del sistema CRUD ampliado, no solo sobre barajas sino también sobre la colección personal del usuario. La inclusión de endpoints de estadística y búsqueda externa aporta un valor añadido al sistema.

Su funcionamiento detallado y configuración se encuentran descritos en el archivo README del repositorio, donde se abordan aspectos como la lógica de interacción, validaciones y comunicación con la base de datos.

4.3. Desarrollo del frontend y flujo operativo real

Paralelamente, el desarrollo de la interfaz de usuario evolucionó desde prototipos estáticos hacia un flujo operativo completo y dinámico. Se implementó un sistema de gestión de sesiones que permite el cambio fluido entre las vistas de registro e inicio de sesión, garantizando que el usuario acceda únicamente a su colección privada almacenada en MongoDB una vez identificado.

Tras la validación del acceso, se desarrollaron las vistas de colección y el detalle de cada baraja. Estas interfaces permiten al usuario interactuar en tiempo real con su contenido, editando la composición del mazo y persistiendo los cambios de forma inmediata con la API. Esta evolución incluye una refactorización integral del cliente por áreas de responsabilidad (autenticación, colección y comunicación con la API) para facilitar la escalabilidad futura.

Como fase de cierre de la interfaz, se incorporó la funcionalidad de borrado definitivo de barajas y se aplicaron técnicas de diseño responsive. Estas mejoras aseguran que la experiencia de usuario sea óptima y visualmente coherente en una amplia variedad de dispositivos.

Figura 7. Pantalla de Login/Registro

Figura 8. Pantalla de Colección de barajas

Figura 9. Pantalla de Mi colección

Figura 10. Pantalla de Crear nueva baraja

Figura 11. Pantalla de Mi baraja

Figura 12. Pantalla de Buscador de cartas

4.4. Capacidades multijuego y editor visual de baraja

Una de las partes más relevantes de esta fase fue la transformación de la plataforma en una solución multijuego auténtica. Se integra el tipo de juego (Pokémon, Magic o Yu-GI-Oh!) como un atributo estructural dentro de la base de datos de MongoDB, permitiendo que el sistema se adapte dinámicamente según la elección del usuario al crear una nueva baraja.

Gracias a esta base polimórfica, se desarrolló un endpoint de búsqueda unificado capaz de consultar múltiples fuentes externas de manera homogénea. Esta integración permite al usuario buscar cartas de diferentes franquicias bajo un

mismo estándar visual, simplificando la complejidad técnica de las diferentes estructuras de datos de origen.

Finalmente, se construyó el editor visual de barajas, la herramienta central de la aplicación. En este componente, los usuarios pueden añadir o retirar cartas mediante controles directos, mientras el sistema aplica automáticamente las reglas de validación específicas de cada juego dentro de la lógica de actualización, garantizando que el mazo resultante sea legal dentro de cada uno de los juegos.

Figura 13. Atributo GameType (MongoDB)

4.5. Resultados de la fase

La Fase 2 concluyó exitosamente con un núcleo funcional operativo que integra autenticación, vinculación de datos por usuario, un CRUD completo y un motor multijuego. El sistema ha demostrado su funcionamiento al permitir la construcción de mazos completos de forma muy clara e interactiva.

Desde el punto de vista arquitectónico, la solución queda preparada para crecer sin necesidad de reestructurar la base de datos para cada nueva franquicia. Se ha logrado una separación clara entre la interfaz, la lógica de negocio y la capa de persistencia, dejando una base sólida para las próximas etapas.

5. Memoria Técnica y Presentación

5.1. Elaboración de la documentación técnica

La tercera fase del proyecto se ha dedicado íntegramente a la consolidación de toda la información generada durante el ciclo de vida del desarrollo para cumplir con los requisitos de evaluación de la asignatura. El núcleo de esta fase es la redacción de la presente memoria, un documento que no solo describe el producto final, sino que justifica cada decisión tomada por el grupo 24, desde la elección del stack tecnológico hasta la implementación de las reglas de negocio con las barajas.

Esta labor de documentación ha permitido realizar una reflexión crítica sobre el trabajo realizado, asegurando que todo lo alcanzado en las fases anteriores quede registrado. La memoria actúa como el soporte principal que demuestra la adquisición de competencias en el uso de C# y MongoDB, detallando cómo se ha resuelto el problema de la variabilidad de datos mediante un sistema CRUD eficiente y escalable.

5.2. Preparación de la presentación

Un componente fundamental de esta etapa ha sido la preparación de la presentación requerida para la asignatura. Este proceso ha consistido en sintetizar los aspectos más relevantes del desarrollo de DeckBuilder para su exposición. Se han diseñado materiales visuales que destacan los puntos fuertes de la aplicación, como la integración de las APIs externas y la flexibilidad de la base de datos documental.

El objetivo de esta tarea es comunicar de forma clara y concisa los resultados obtenidos y demostrar que el núcleo funcional desarrollado en la fase anterior es operativo y responde a los objetivos planteados al inicio.

5.3. Desarrollo del archivo README: Manual usuario y despliegue

Como cierre técnico del proyecto, se ha elaborado un archivo README dentro del repositorio del código fuente. Este documento se ha diseñado para cumplir una doble función: servir como manual de usuario y como guía detallada de despliegue e instalación. En él se explican paso a paso las instrucciones necesarias para que cualquier usuario pueda configurar el entorno de ejecución, incluyendo la vinculación con MongoDB e instalación de dependencias de .NET 8.

Además de las instrucciones técnicas, el README incluye una guía de uso que describe el flujo operativo de la aplicación.

5.4. Resultados finales de la fase

La conclusión de esta fase marca el fin del desarrollo de DeckBuilder. Con la memoria técnica finalizada, la presentación preparada y el archivo README configurado, el proyecto queda listo para su entrega definitiva. El resultado es una aplicación que funciona de forma correcta, respaldada por una documentación para entender todo lo realizado.

6. Conclusiones

Tras la finalización del desarrollo de DeckBuilder, se ha logrado cumplir el objetivo principal de diseñar e implementar una aplicación funcional que utilice las tecnologías de C# y MongoDB.

Uno de los aspectos más relevantes del proyecto ha sido la utilización de una base de datos documental. MongoDB ha permitido gestionar de una manera eficiente la variabilidad entre los distintos juegos de cartas que hemos usado, evitando de esta forma las limitaciones de los modelos relacionales que conocemos.

Desde el punto de vista técnico, el desarrollo de una API REST completa, junto con un frontend interactivo, ha permitido cubrir todo el ciclo de vida del usuario a lo largo de la aplicación, desde la autenticación hasta la gestión avanzada de barajas y colecciones. La integración con APIs externas aporta un valor añadido y significativo a la aplicación.

En conclusión, el proyecto ha permitido aplicar de forma práctica los conceptos estudiados en la asignatura, especialmente en el uso de bases de datos NoSQL, consolidando de esta forma una base sólida tanto a nivel técnico como a nivel conceptual.

Bibliografía

- [1] Agustín Borrego, Daniel Ayala, Inma Hernández, Carlos R. Rivero, and David Ruiz. Generating rules to filter candidate triples for their correctness checking by knowledge graph completion techniques. In *K-CAP*, pages 115–122, 2019. doi: 10.1145/3360901.3364418.
- [2] Universidad de Sevilla. Página principal de la escuela técnica de ingeniería informática, 2020. URL <https://www.informatica.us.es>.
- [3] Agustín Borrego, Daniel Ayala, Inma Hernández, Carlos R. Rivero, and David Ruiz. CAFE: Knowledge graph completion using neighborhood-aware features. *Eng. Appl. Artif. Intell.*, 103:104302, 2021. doi: 10.1016/j.engappai.2021.104302. URL <https://doi.org/10.1016/j.engappai.2021.104302>.